



AMRITA

VISHWA VIDYAPEETHAM

SUBMITTED BY:

NAME:	M. Sowmya Sree
ROLL NO:	CSE25040
YEAR / SEM / SECTION	2 / 3 / CSE-A
CREDITS	5

SUBMITTED TO:

NAME:	Dr. RAJKUMAR BATCHU
DEPARTMENT	SCHOOL OF COMPUTING
DESIGNATION	ASST. PROFESSOR (SI grade)

S NO:	TITLE	PAGE	REMARKS
1	<u>Lab-1:</u> 1.A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion .write a recursion python program to display a countdown from n to 1 followed by “launch”. 2.Banks calculate the growth of an investment using compound interest the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power),where p=principal growth factor, n= number of years 3. An HR department stores employee ID’s in a list. Develop a recursive program to search for a given employee ID .write a recursive function to search for an employee ID in a list. 4. In a logistics company the number of possible ways to arrange n different parcels is calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number. 5. write a recursive python program to print the first n terms of the Fibonacci series.	5-12	
2	<u>Lab-2:</u> 1.Write a python program to perform linear search on a given input. 2. Write a Python program to perform binary search on the given input. Verify by taking sorted array & unsorted array as input.	13-16	
3	<u>Lab-3:</u> 1. Write a Python program to sort the elements using a bubble sort technique. 2. Write a Python program to sort the elements using a insertion sort technique. 3. Write a Python program to sort the elements using a selection sort technique. 4.Write a Python program to sort the elements using a merge sort technique. 5.Write a Python program to sort the elements using a quick sort technique.	17-23	

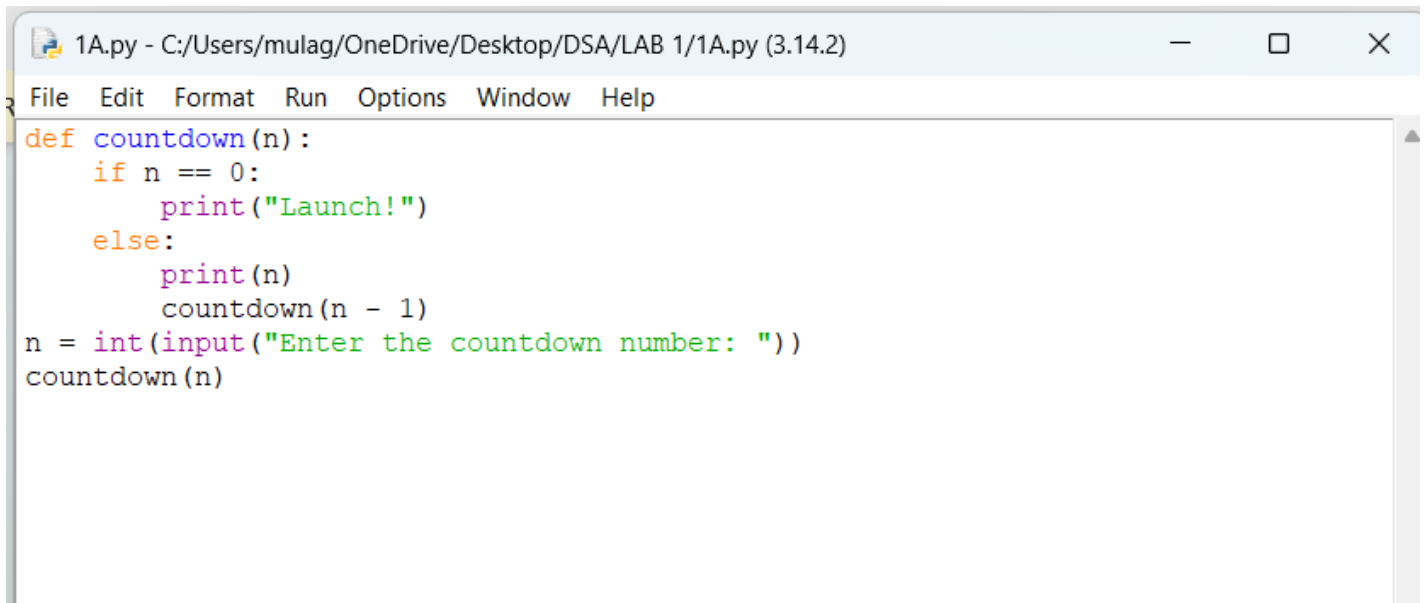
4	<u>Lab-4:</u> 1). Write a python program to implement a SLL which could perform the below operations by taking an input from the user. 1) create a LL 2) Insert at beginning 3) Insert at the end 4) Insert at specific position 5) delete by value 6) delete first node 7) delete last node 8) Count no. of nodes 9) display / traverse 10) exit 2).Write a python program to implement a DLL which could perform the below operations by taking an input from the user. 1) create a DLL 2) Insert at beginning 3) Insert at the end 4) Insert at specific position 5) deletion at beginning 6) deletion at the end 7) deletion at the specific position 8) Count no. of nodes 9) display / traverse 10) exit 3). AIM: Write a python program to implement a CLL which could perform the below operations by taking an input from the user. 1) create a CLL 2) Insert at beginning 3) Insert at the end 4) Insert at specific position 5) deletion at beginning 6) deletion at the end 7) deletion at specific position 8) Count no. of nodes 9) display / traverse 10) display head and tail 11) exit	24 -48	
---	---	--------	--

5	<u>Lab-5:</u> 1. Write a python program to implement a stack using an Array. Implement the stack operations such as push, pop, peak and also display the stack elements. 2. Write a python program to implement a stack using an Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.	49-55	

Lab-1

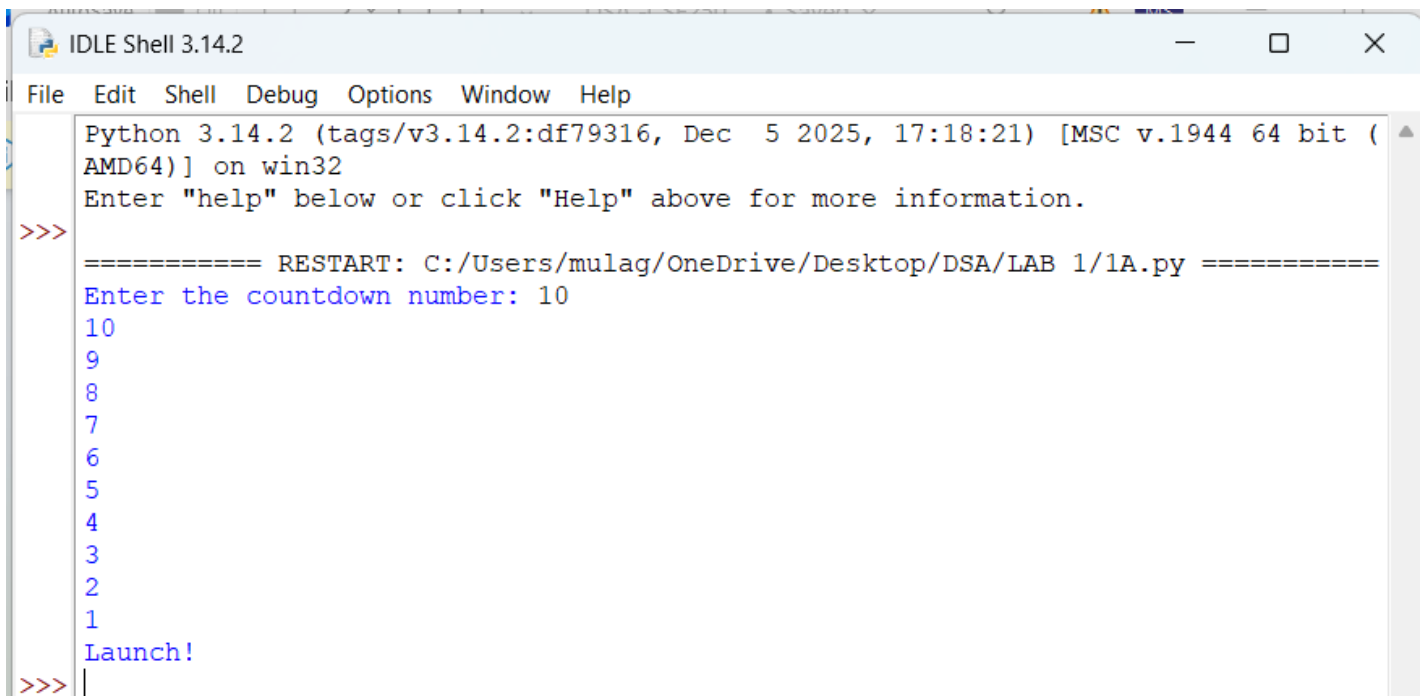
1)AIM: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion .write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

A screenshot of a Python IDE window titled '1A.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1A.py (3.14.2)'. The window contains a Python program that defines a recursive function 'countdown(n)'. The function checks if 'n' is 0; if so, it prints 'Launch!'. Otherwise, it prints 'n' and calls itself with 'n - 1'. The program prompts the user to 'Enter the countdown number:' and then calls the 'countdown' function with the input value.

```
def countdown(n):  
    if n == 0:  
        print("Launch!")  
    else:  
        print(n)  
        countdown(n - 1)  
n = int(input("Enter the countdown number: "))  
countdown(n)
```

OUTPUT:

A screenshot of an IDLE Shell window titled 'IDLE Shell 3.14.2'. It shows the execution of the program. The prompt '>>>' is followed by the program's output: a series of numbers from 10 down to 1, followed by 'Launch!'. The prompt '>>>' is shown again at the bottom.

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>>  
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1A.py =====  
Enter the countdown number: 10  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
Launch!  
>>>
```

EXPLANATION:

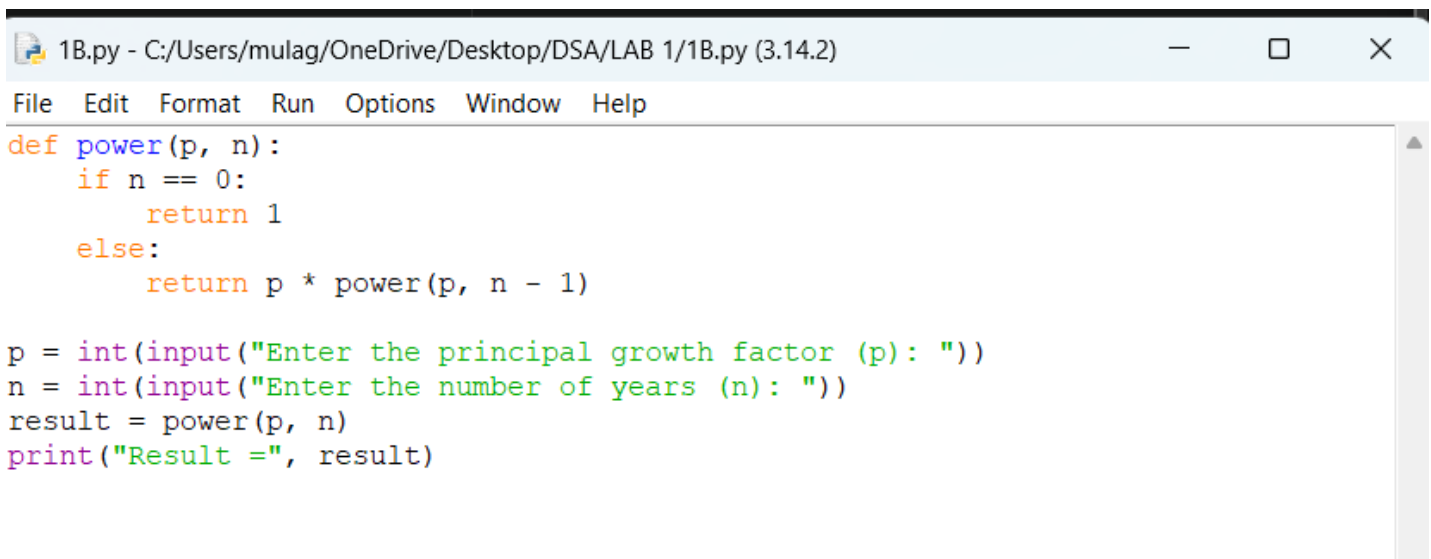
This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

S.NO	ERROR	ERROR RECTIFICATION
1	Indentation Error	Align blocks properly
2	Value Error: invalid literal for int()	Enter a valid integer
3	Name Error: name 'countdown' is not defined	Use the correct function name: countdown().

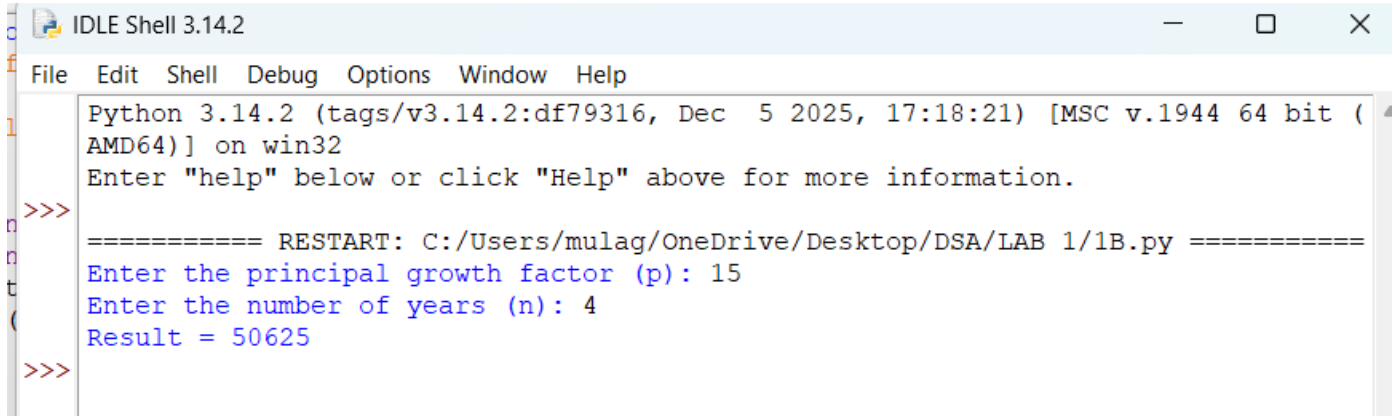
2) AIM: Banks calculate the growth of an investment using compound interest the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n (power), where p =principal growth factor, n = number of years.

PROGRAM:

A screenshot of a Python IDE window titled '1B.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1B.py (3.14.2)'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor contains the following Python code:

```
def power(p, n):  
    if n == 0:  
        return 1  
    else:  
        return p * power(p, n - 1)  
  
p = int(input("Enter the principal growth factor (p): "))  
n = int(input("Enter the number of years (n): "))  
result = power(p, n)  
print("Result =", result)
```

OUTPUT:



```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1B.py =====
Enter the principal growth factor (p): 15
Enter the number of years (n): 4
Result = 50625
>>>
```

EXPLANATION:

This program recursively calculates p^n by multiplying p until n becomes 0.

ERROR TABLE:

<u>S NO</u>	<u>ERROR</u>	<u>ERROR RECTIFICATION</u>
<u>1</u>	Name Error – Function name is spelled incorrectly.	Use the correct function name
<u>2</u>	Value Error – entered letters instead of a number	Enter a valid integer
<u>3</u>	Syntax Error – Missing : after if or else.	Add the colon (:) after if and else statements.

3)AIM:

An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID .write a recursive function to search for an employee ID in a list.

PROGRAM:

```

File Edit Format Run Options Window Help
def search_emp(emp_list, key, index):
    if index >= len(emp_list):
        return False
    if emp_list[index] == key:
        return True
    return search_emp(emp_list, key, index + 1)

emp_list = [101, 102, 103, 104, 105]

key = int(input("Enter Employee ID to search: "))
if search_emp(emp_list, key, 0):
    print("Employee ID Found")
else:
    print("Employee ID Not Found")

```

OUTPUT:

```

File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1C.py =====
Enter Employee ID to search: 101
Employee ID Found
>>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1C.py =====
Enter Employee ID to search: 2327
Employee ID Not Found
>>>

```

EXPLANATION: This program searches for an employee ID in a list and returns its index if found.

ERROR TABLE:

S.NO	ERRORS	ERRORS RECTIFICATION
1	Name Error – Function name is typed incorrectly.	Use the correct function name search_emp()
2	Value Error – User enters letters instead of a number.	Enter a valid employee ID (integer), e.g., 103.
3	Syntax Error – Missing : after if statement.	Add : after if and else statements.

4)AIM: In a logistics company the number of possible ways to arrange n different parcels is calculated using the factorial formula($n!$). develop a recursive program to calculate the factorial of a given number

PROGRAM:

```
1D.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1D.py (3.14.2)
File Edit Format Run Options Window Help
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
n = int(input("Enter a number: "))

if n < 0:
    print("Invalid input! Factorial is not defined for negative numbers.")
else:
    print("Factorial =", factorial(n))
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1D.py =====
Enter a number: 5
Factorial = 120
>>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1D.py =====
Enter a number: -1
Invalid input! Factorial is not defined for negative numbers.
>>> |
```

EXPLANATION:

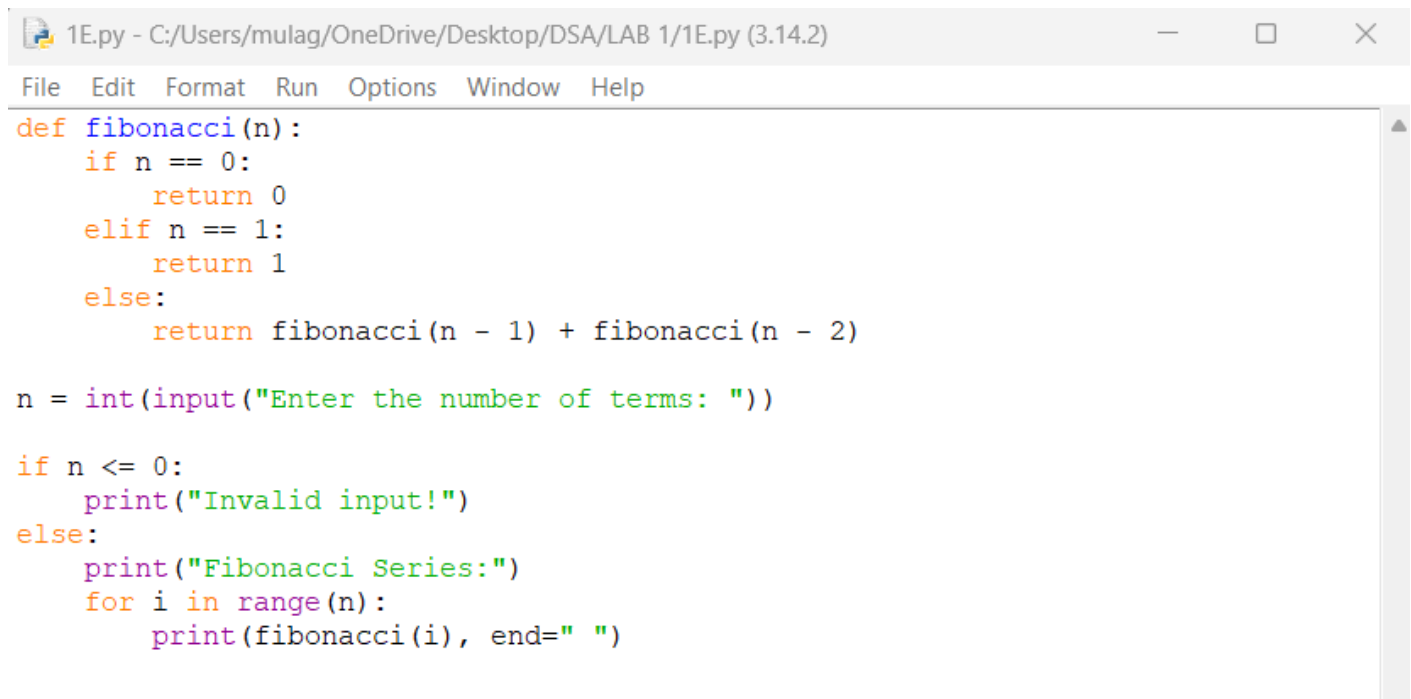
This program calculates the factorial of a number using iteration (not recursion).

ERROR TABLE:

SNO	ERRORS	ERRORS RECTIFICATION
1	Overflow/large input	Give smaller input
2	Name Error – Function name is typed incorrectly.	Use the correct function name factorial().

5)AIM: write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

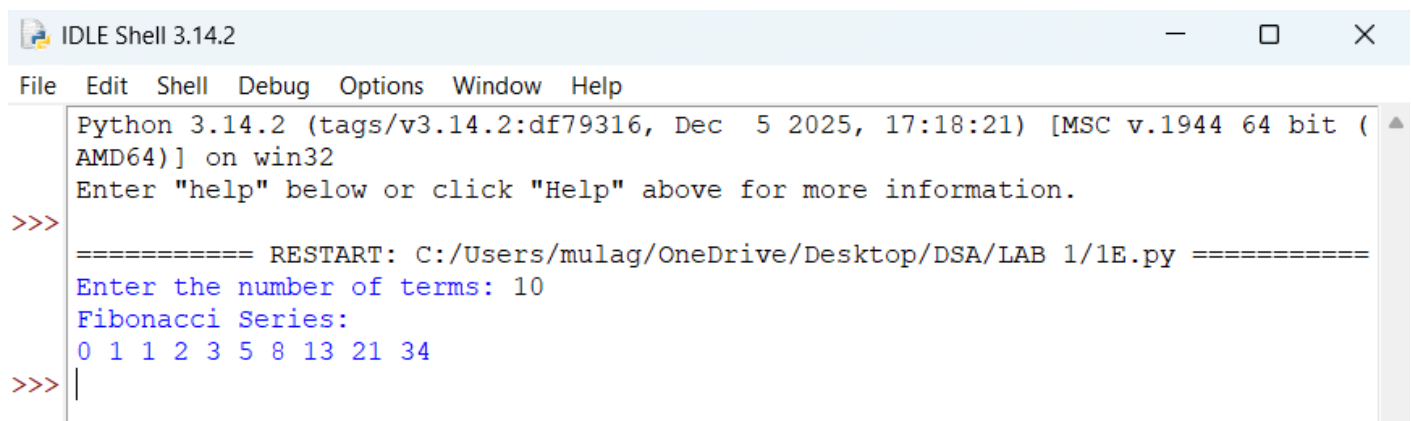


```
1E.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1E.py (3.14.2)
File Edit Format Run Options Window Help
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter the number of terms: "))

if n <= 0:
    print("Invalid input!")
else:
    print("Fibonacci Series:")
    for i in range(n):
        print(fibonacci(i), end=" ")
```

OUTPUT:



```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1E.py =====
Enter the number of terms: 10
Fibonacci Series:
0 1 1 2 3 5 8 13 21 34
>>> |
```

EXPLANATION:

The program uses recursion to generate Fibonacci numbers. $\text{fib}(0)=0$ and $\text{fib}(1)=1$ are the base cases. For every other term, $\text{fib}(n)=\text{fib}(n-1) + \text{fib}(n-2)$. The loop prints the first n terms.

ERROR TABLE:

SNO	ERRORS	ERRORS RECTIFICATION
1	Recursion Error – Base case is missing or incorrect in the recursive function.	Add the correct base cases: if $n == 0$: return 0 and elif $n == 1$: return 1.
2	Indentation Error – Incorrect indentation after if or else.	Indent the code properly using spaces or tabs.
3	Type Error – Passing a string instead of an integer.	Convert the input using <code>int(input())</code> .

PROGRAM	TIME COMPLEXITY	SPACE COMPLEXITY	REASON
1.countdown to launch	O(n)	O(1)	The program uses a loop that prints from n to 0 once.
2.power calculation	O(n)	O(n)	One recursive call is made for each value of n . The recursion stack stores n calls.
3.employee ID search	O(n)	O(1)	The program performs a linear search through the list.
4.Factorial	O(n)	O(1)	The program calculates factorial using a loop from 1 to n .
5.Fibonacci series	O(2ⁿ)	O(n)	Each function call creates two more recursive calls, leading to exponential time. The recursion stack reaches depth n .

Lab-2

1)Aim: Write a python program to perform linear search on a given input.

Program:

```
2A.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2A.py (3.14.2)
File Edit Format Run Options Window Help
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

n=int(input("Enter the no of elements: "))
arr=[]
print("enter the elements:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter element to search: "))

result = linear_search(arr, key)

if result != -1:
    print("Element found at position", result)
else:
    print("Element not found")
```

Output:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
.>>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2A.py =====
Enter the no of elements: 6
enter the elements:
3
5
9
8
9
1
Enter element to search: 1
Element found at position 5
.>>> |
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Takes the target element to be searched.
- Linear Search() checks each element one by one.
- If the target is found, its index is returned.
- If not found, -1 is returned.
- The result is displayed using print().

ERROR TABLE:

S.No	Error Obtained	Rectification
<u>1</u>	Element is not found	Check whether the required element is present in the list
<u>2</u>	Wrong index is displayed	Check the list indexing starting from 0
<u>3</u>	Search stops unexpectedly	Verify the range used in the for loop

2. Aim:

Write a Python program to perform binary search on the given input. Verify by taking sorted array & unsorted array as input.

Program:

2b.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2b.py (3.14.2)

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

n = int(input("Enter number of elements: "))
arr = []
print("Enter", n, "elements:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at position", result + 1)
else:
    print("Element not found")
```

Output:

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2b.py =====
Enter number of elements: 5
Enter 5 elements:
10
20
30
40
50
Enter element to search: 40
Element found at position 4
>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2b.py =====
Enter number of elements: 5
Enter 5 elements:
50
10
30
20
40
Enter element to search: 40
Element found at position 5
>> |
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Sorts the list using `a.sort()` because binary search requires sorted data.
- Calculates the middle element and compares it with the target.
- If the target is smaller or larger, searches only the appropriate half.
- Returns the index if found; otherwise returns -1.
- Displays whether the element is found or not.

ERROR TABLE:

S.No	Error Obtained	Error Rectification
1	Wrong element is returned	Make sure the list is sorted before searching
2	Search gives incorrect index	Check the calculation of the middle index
3	Element is not found	Verify the low and high boundaries

Lab-3

1.Aim: Write a Python program to sort the elements using a Bubble sort technique.

Program:

```
3.1.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.1.py (3.14.2)
File Edit Format Run Options Window Help
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)
bubble_sort(arr)
print("Sorted elements:", arr)
```

Output:

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.1.py =====
Enter the number of elements: 5
Enter element 1: 40
Enter element 2: 29
Enter element 3: 44
Enter element 4: 35
Enter element 5: 23
Sorted elements: [23, 29, 35, 40, 44]
>>> |
```

Explanation:

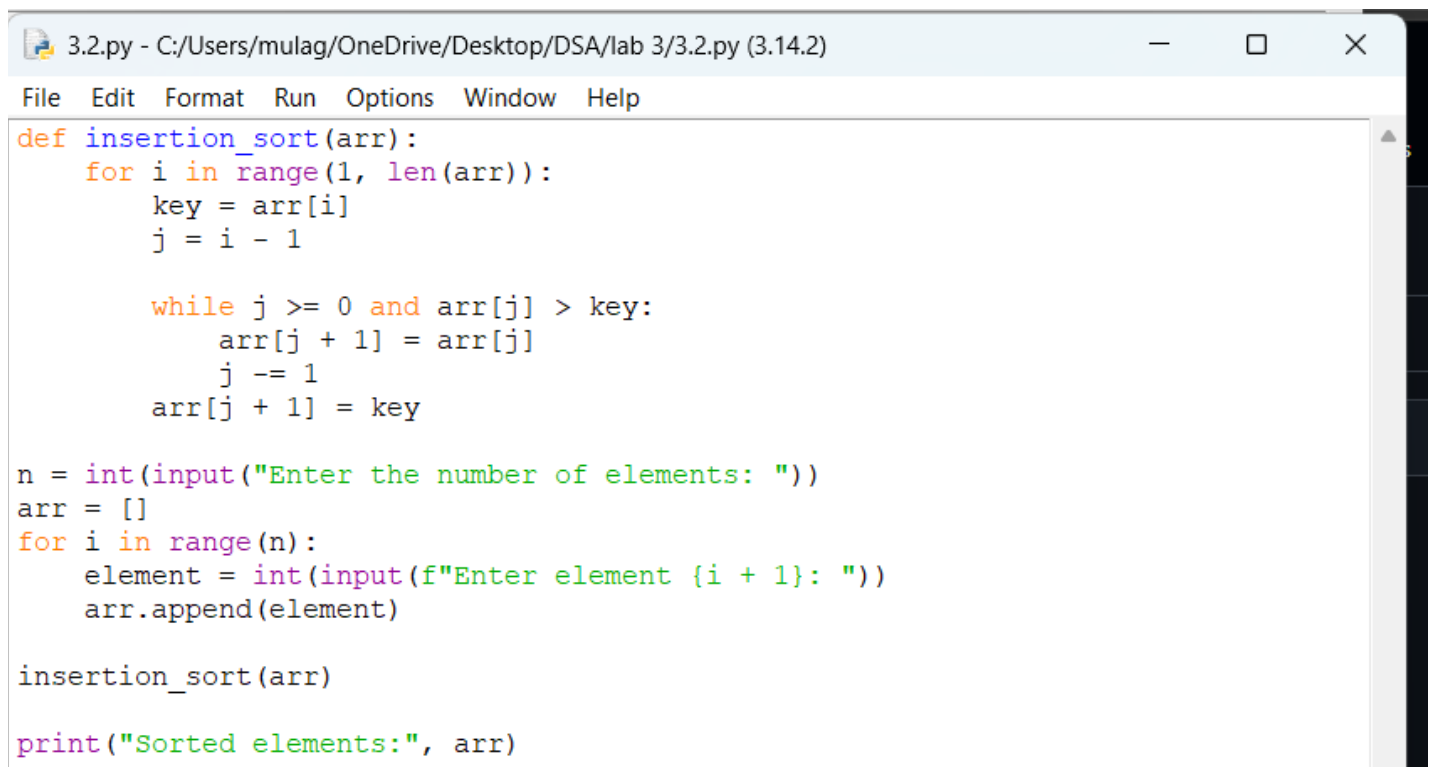
Bubble Sort compares two adjacent elements. If the first element is bigger than the second, it swaps them. This process repeats until all elements are arranged in ascending order.

Error table:

S.No	Error Obtained	Error Rectification
1	Name Error because the function name is incorrect	Check that bubble_sort(arr) is written correctly
2	Value Error when entering input	Enter only integer values

2.Aim: Write a Python program to sort the elements using a insertion sort technique.

Program:



```
3.2.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.2.py (3.14.2)
File Edit Format Run Options Window Help
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

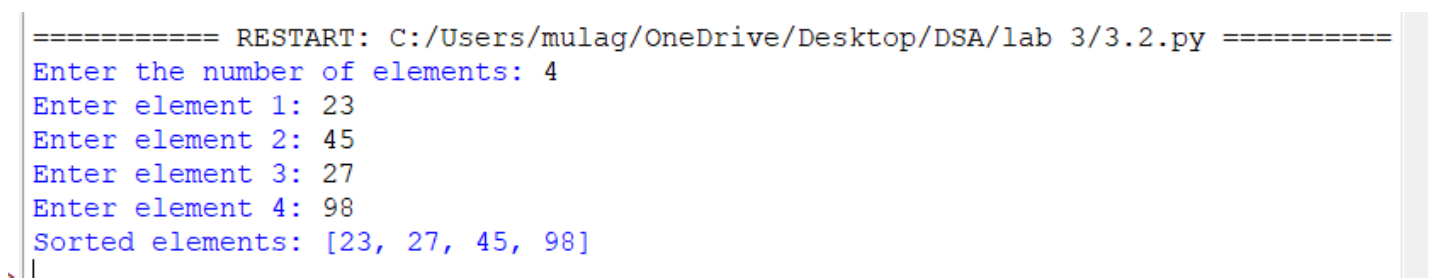
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)

insertion_sort(arr)

print("Sorted elements:", arr)
```

Output:



```
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.2.py =====
Enter the number of elements: 4
Enter element 1: 23
Enter element 2: 45
Enter element 3: 27
Enter element 4: 98
Sorted elements: [23, 27, 45, 98]
>|
```

Explanation:

This program sorts the given elements in **ascending order using Insertion Sort**. It takes each element (key) and compares it with the elements before it, shifting larger elements one position to the right. The correct position for the key is then found and the element is inserted there. Finally, the sorted list is displayed using print().

Error table:

<u>S.No</u>	<u>Error Obtained</u>	<u>Error Rectification</u>
<u>1</u>	Syntax Error due to missing colon (:)	Add : after def, for, and while statements
<u>2</u>	Indentation Error due to incorrect spacing	Maintain proper indentation inside loops and functions

3.Aim: Write a Python program to sort the elements using a selection sort technique.

Program:

```
3.3.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.3.py (3.14.2)
File Edit Format Run Options Window Help
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)
selection_sort(arr)
print("Sorted elements:", arr)
```

Output:

```
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.3.py =====
Enter the number of elements: 4
Enter element 1: 67
Enter element 2: 89
Enter element 3: 34
Enter element 4: 23
Sorted elements: [23, 34, 67, 89]
|
```

Explanation:

This program sorts the given elements in **ascending order using Selection Sort**. It finds the smallest element from the unsorted part of the list and swaps it with the current position. This process repeats until all elements are sorted, and finally the sorted list is displayed.

Error table:

<u>S.No</u>	<u>Error Obtained</u>	<u>Error Rectification</u>
1.	Syntax Error due to missing colon (:)	Add : after def, for, and if statements
2.	Indentation Error due to spacing	Maintain proper indentation inside the function and loops
3.	Name Error due to wrong variable name	Check the spelling of variables like arr, min_index, i, and j

4.Aim: Write a Python program to sort the elements using a merge sort technique.

Program:

```

def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)
        i = 0
        j = 0
        k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)

merge_sort(arr)

print("Sorted elements:", arr)

```

Output:

```

===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.4.py =====
Enter the number of elements: 6
Enter element 1: 34
Enter element 2: 76
Enter element 3: 89
Enter element 4: 43
Enter element 5: 23
Enter element 6: 27
Sorted elements: [23, 27, 34, 43, 76, 89]

```

Explanation:

This program sorts the given elements in **ascending order using Merge Sort**. It divides the list into two halves repeatedly, sorts each half using recursion, and then merges them in the correct order. Finally, the sorted elements are displayed.

Error table:

S.No	Error Obtained	Error Rectification
1	Type Error while entering invalid input	Enter only integer values
2	Name Error because merge_sort is misspelled	Use the same function name merge_sort(arr) everywhere

5.Aim: Write a Python program to sort the elements using a quick sort technique.

Program:

```
3.5.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.5.py (3.14.2)
File Edit Format Run Options Window Help
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    left = []
    right = []
    for i in arr[:-1]:
        if i < pivot:
            left.append(i)
        else:
            right.append(i)
    return quick_sort(left) + [pivot] + quick_sort(right)
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)

arr = quick_sort(arr)
print("Sorted elements:", arr)
```

Output:

```
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.5.py =====
Enter the number of elements: 5
Enter element 1: 33
Enter element 2: 44
Enter element 3: 66
Enter element 4: 22
Enter element 5: 11
Sorted elements: [11, 22, 33, 44, 66]
```

Explanation:

This program sorts the given elements in **ascending order using Quick Sort**. It selects the last element as the **pivot**, places smaller elements in the left list and larger or equal elements in the right list, then recursively sorts both lists. Finally, it combines the sorted lists and displays the result.

Error table:

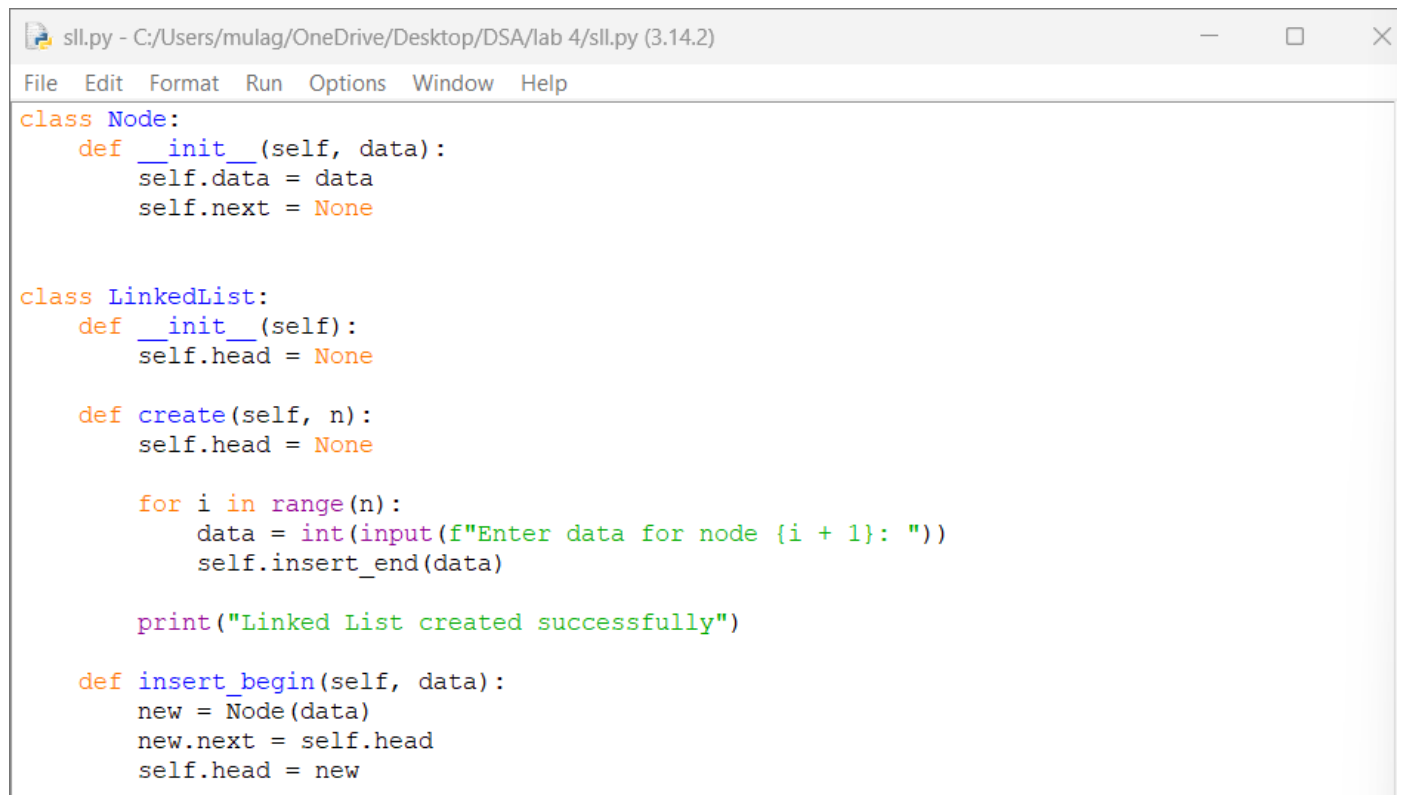
S.No	Error Obtained	Error Rectification
1	Type Error while entering invalid input	Enter only integer values
2	Recursion Error due to incorrect recursive calls	Ensure the base condition if <code>len(arr) <= 1</code> : is present

Lab-4

1. AIM: Write a python program to implement a SLL which could perform the below operations by taking an input from the user.

- 1) create a LL
- 2) Insert at beginning
- 3) Insert at the end
- 4) Insert at specific position
- 5) delete by value
- 6) delete first node
- 7) delete last node
- 8) Count no. of nodes
- 9) display / traverse
- 10) exit

PROGRAM:

A screenshot of a Python IDE window titled 'sll.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 4/sll.py (3.14.2)'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code is as follows:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def create(self, n):
        self.head = None

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            self.insert_end(data)

        print("Linked List created successfully")

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new
```

```
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
    else:
        temp = self.head

        while temp.next:
            temp = temp.next

        temp.next = new

def insert_position(self, data, position):
    new = Node(data)

    if position < 1:
        print("Invalid position")
        return

    if position == 1:
        new.next = self.head
        self.head = new
        print("Node inserted successfully")
        return
    temp = self.head

    for i in range(position - 2):
        if temp is None:
            print("Invalid position")
            return
        temp = temp.next

    if temp is None:
        print("Invalid position")
    else:
        new.next = temp.next
        temp.next = new
        print("Node inserted successfully")

def delete_value(self, value):
    if self.head is None:
        print("No Data to delete")
        return
```

```
if self.head.data == value:
    self.head = self.head.next
    print("Value deleted")
    return

temp = self.head

while temp.next and temp.next.data != value:
    temp = temp.next

if temp.next is None:
    print("Value not present")
else:
    temp.next = temp.next.next
    print("Value deleted")

def delete_first(self):
    if self.head is None:
        print("No Data to delete")
    else:
        print("Deleted Value =", self.head.data)
        self.head = self.head.next

def delete_last(self):
    if self.head is None:
        print("No Data to delete")

    elif self.head.next is None:
        print("Deleted Value =", self.head.data)
        self.head = None

    else:
        temp = self.head

        while temp.next.next:
            temp = temp.next

        print("Deleted Value =", temp.next.data)
        temp.next = None

def count(self):
    if self.head is None:
        print("No Linked List")
    else:
        c = 0
```

File Edit Format Run Options Window Help

```
        c = 0
        temp = self.head

        while temp:
            c += 1
            temp = temp.next

        print("Number of nodes =", c)

    def display(self):
        if self.head is None:
            print("No Linked List")
        else:
            temp = self.head

            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next

            print("None")

ll = LinkedList()

while True:
    print("\n--- Singly Linked List Operations ---")
    print("1. Create a Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specific Position")
    print("5. Delete a Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display Nodes")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        n = int(input("Enter number of nodes: "))
        ll.create(n)

    elif choice == 2:
        data = int(input("Enter data: "))
        ll.insert_begin(data)
```

```
--- Singly Linked List Operations ---
```

1. Create a Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Position
5. Delete a Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display Nodes
10. Exit

```
Enter your choice: 1
```

```
Enter number of nodes: 4
```

```
Enter data for node 1: 1
```

```
Enter data for node 2: 5
```

```
Enter data for node 3: 7
```

```
Enter data for node 4: 8
```

```
Linked List created successfully
```

```
--- Singly Linked List Operations ---
```

1. Create a Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Position
5. Delete a Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display Nodes
10. Exit

```
Enter your choice: 8
```

```
Number of nodes = 4
```

```
--- Singly Linked List Operations ---
```

1. Create a Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Position
5. Delete a Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display Nodes
10. Exit

```
Enter your choice: 10
```

```
Program ended
```

```
>>> |
```

EXPLANATION:

1. class Node

- Defines a node of the Singly Linked List.
- self.data stores the value of the node.
- self.next stores the reference to the next node.

2. class LinkedList

- Defines the Linked List and its operations.
- self.head points to the first node of the list.

3. create()

- Creates n nodes using a for loop.
- Takes data from the user and calls insert_end() to add each node.

4. insert_begin()

- Creates a new node using Node(data).
- new.next = self.head links it to the current first node.
- Updates self.head to make the new node the first node.

5. insert_end()

- Creates a new node.
- Uses while temp.next to traverse to the last node.
- Connects the new node using temp.next = new.

6. insert_position()

- Inserts a node at a specific position.
- Traverses to the required location using a loop.
- Updates the next references to connect the new node correctly.

7. delete_value()

- Searches for a node containing the given value.
- Uses a while loop for traversal.
- Deletes the node by changing the appropriate next reference.

8. delete_first()

- Deletes the first node.
- Updates the head using: `self.head = self.head.next`.

9. delete_last()

- Traverses to the second-last node.
- Sets `temp.next = None` to remove the last node.
- Also handles the special case of a single-node list.

10. count()

- Traverses the linked list using `temp`.
- Increments counter `c` for every node.
- Displays the total number of nodes.

11. display()

- Traverses the list from head until `None`.
- Prints the data of each node in linked list format.

12. Main Program

- `LL = LinkedList()` creates an object of the `LinkedList` class.
- `while True` creates an infinite loop to repeatedly display the menu.
- `if-elif` statements perform operations based on the user's choice.
- `break` terminates the loop and ends the program when `Exit` is selected.

ERROR TABLE:

s.no	Error Obtained	Rectification
1	Invalid insertion position.	Added position validation using if position < 1 and if temp is None.
2	Value to be deleted may not exist.	Checked using if temp.next is None and displayed "Value not present".
3	Program may terminate unexpectedly during normal operations.	Used proper conditional statements and return statements to control program flow.

2. AIM: Write a python program to implement a DLL which could perform the below operations by taking an input from the user.

- 1) create a DLL
- 2) Insert at beginning
- 3) Insert at the end
- 4) Insert at specific position
- 5) deletion at beginning
- 6) deletion at the end
- 7) deletion at the specific position
- 8) Count no. of nodes
- 9) display / traverse
- 10) exit

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node
                new_node.prev = temp
            print("Linked List created successfully.")

    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
        print("Node inserted at beginning.")

    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)
```

```
    if self.head is None:
        self.head = new_node
    else:
        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    print("Node inserted at end.")

def insert_position(self):
    position = int(input("Enter position: "))
    data = int(input("Enter data: "))

    if position <= 0:
        print("Invalid position.")
        return

    if position == 1:
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node
        print("Node inserted.")
        return

    temp = self.head

    for i in range(1, position - 1):
        if temp is None:
            print("Invalid position.")
            return
        temp = temp.next

    if temp is None:
        print("Invalid position.")
        return

    new_node = Node(data)

    new_node.next = temp.next
```

```
new_node.next = temp.next
new_node.prev = temp

if temp.next is not None:
    temp.next.prev = new_node

temp.next = new_node

print("Node inserted.")

def delete_beginning(self):
    if self.head is None:
        print("Linked List is empty.")
        return

    self.head = self.head.next

    if self.head is not None:
        self.head.prev = None

    print("Node deleted from beginning.")

def delete_end(self):
    if self.head is None:
        print("Linked List is empty.")
        return

    temp = self.head

    if temp.next is None:
        self.head = None
        print("Node deleted from end.")
        return

    while temp.next is not None:
        temp = temp.next

    temp.prev.next = None

    print("Node deleted from end.")

def delete_position(self):
    position = int(input("Enter position: "))

    if self.head is None:
        print("Linked List is empty.")
        return
```

```
        return

    if position == 1:
        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("Node deleted.")
        return

    temp = self.head

    for i in range(1, position):
        if temp is None:
            print("Invalid position.")
            return
        temp = temp.next

    if temp is None:
        print("Invalid position.")
        return

    if temp.next is not None:
        temp.next.prev = temp.prev

    if temp.prev is not None:
        temp.prev.next = temp.next

    print("Node deleted.")

def count(self):
    count = 0
    temp = self.head

    while temp is not None:
        count += 1
        temp = temp.next
    print("Number of nodes:", count)

def display(self):
    if self.head is None:
        print("Linked List is empty.")
        return
    temp = self.head
    print("Doubly Linked List:")
    while temp is not None:
```

```
while temp is not None:
    print(temp.data, end=" <-> ")
    temp = temp.next

    print("None")
```

```
dll = DoublyLinkedList()
```

```
while True:
```

```
    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Create a LL")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific position")
    print("5. Deletion at beginning")
    print("6. Deletion at end")
    print("7. Deletion at specific position")
    print("8. Count no. of nodes")
    print("9. Display / Traverse")
    print("10. Exit")
```

```
    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        dll.create()
```

```
    elif choice == 2:
        dll.insert_beginning()
```

```
    elif choice == 3:
        dll.insert_end()
```

```
    elif choice == 4:
        dll.insert_position()
```

```
    elif choice == 5:
        dll.delete_beginning()
```

```
    elif choice == 6:
        dll.delete_end()
```

```
    elif choice == 7:
        dll.delete_position()
```

```
    elif choice == 8:
        dll.count()
```

```

        return 0;

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Program exited.")
        break

    else:
        print("Invalid choice. Please try again.")

```

Output:

```

===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/ll:

```

```

----- DOUBLY LINKED LIST -----
1. Create a LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Deletion at beginning
6. Deletion at end
7. Deletion at specific position
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 30
Enter data: 23
Enter data: 27
Linked List created successfully.

```

```

----- DOUBLY LINKED LIST -----
1. Create a LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Deletion at beginning
6. Deletion at end
7. Deletion at specific position
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 10
Node inserted at end.

```

```

Node inserted at end.

----- DOUBLY LINKED LIST -----
1. Create a LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Deletion at beginning
6. Deletion at end
7. Deletion at specific position
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.

```

EXPLANATION:

- The program first defines a Node class to create nodes and initializes data, prev, and next for each node.
- The Doubly LinkedList class initializes an empty list by setting head to None.
- In create(), the user enters the number of nodes and their data. Each new node is added to the end of the list and connected in both directions.
- In insert_beginning(), a new node is created and connected before the current first node. The head is then changed to the new node.
- In insert_end(), the program starts from head, traverses to the last node, and attaches the new node after it.
- In insert_position(), the program traverses to the required position and places the new node between two existing nodes by updating their connections.
- In delete_beginning(), the first node is removed by moving head to the next node and making its prev pointer None.
- In delete_end(), the program traverses to the last node and removes it by disconnecting it from its previous node.
- In delete_position(), the program reaches the node at the given position and connects its previous and next nodes directly, removing the selected node.

- In count(), the program starts from head, visits every node using next, and increments count until the end is reached.
- In display(), every node is visited from the first node to the last node and its data is printed in the form 10 <-> 20 <-> 30.
- The main while True loop repeatedly displays all available operations and takes the user's choice.
- The if-elif statements call the appropriate function according to the selected option.
- When the user enters 10, the break statement stops the loop and terminates the program.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	AttributeError: 'None Type' object has no attribute 'next' may occur when trying to access temp.next while the list is empty.	Check whether self.head or temp is None before accessing next.
2.	Incorrect linking may occur during insertion or deletion if only one of the prev or next pointers is updated.	Update both prev and next pointers whenever a node is inserted or deleted.

3. AIM: Write a python program to implement a CLL which could perform the below operations by taking an input from the user.

- 1) create a CLL
- 2) Insert at beginning
- 3) Insert at the end
- 4) Insert at specific position
- 5) deletion at beginning

- 6) deletion at the end
- 7) deletion at specific position
- 8) Count no. of nodes
- 9) display / traverse
- 10) display head and tail
- 11) exit

PROGRAM:

```
cll.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 4/cll.py (3.14.2)
File Edit Format Run Options Window Help

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        self.head = None
        self.tail = None

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                self.tail = new_node
                new_node.next = self.head
            else:
                new_node.next = self.head
                self.tail.next = new_node
                self.tail = new_node

        print("Circular Linked List created successfully.")

    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

        print("Node inserted at beginning.")
```

```
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        self.tail = new_node
        new_node.next = self.head
    else:
        new_node.next = self.head
        self.tail.next = new_node
        self.tail = new_node

    print("Node inserted at end.")

def insert_position(self):
    position = int(input("Enter position: "))
    data = int(input("Enter data: "))

    if position <= 0:
        print("Invalid position.")
        return

    if self.head is None:
        if position == 1:
            new_node = Node(data)
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
            print("Node inserted.")
        else:
            print("Invalid position.")
            return

    if position == 1:
        self.insert_beginning()
        return

    temp = self.head

    for i in range(1, position - 1):
        temp = temp.next

    if temp == self.head:
        print("Invalid position.")
        return
```

```

        return

    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node

    if temp == self.tail:
        self.tail = new_node

    print("Node inserted.")

def delete_beginning(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return

    if self.head == self.tail:
        print("Deleted Value =", self.head.data)
        self.head = None
        self.tail = None
    else:
        print("Deleted Value =", self.head.data)
        self.head = self.head.next
        self.tail.next = self.head

def delete_end(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return

    if self.head == self.tail:
        print("Deleted Value =", self.tail.data)
        self.head = None
        self.tail = None
        return

    temp = self.head

    while temp.next != self.tail:
        temp = temp.next

    print("Deleted Value =", self.tail.data)

    temp.next = self.head
    self.tail = temp

```

```
def delete_position(self):
    position = int(input("Enter position: "))

    if self.head is None:
        print("Circular Linked List is empty.")
        return
    if position <= 0:
        print("Invalid position.")
        return

    if position == 1:
        self.delete_beginning()
        return

    temp = self.head

    for i in range(1, position - 1):
        temp = temp.next

        if temp == self.head:
            print("Invalid position.")
            return

    if temp.next == self.head:
        print("Invalid position.")
        return

    if temp.next == self.tail:
        print("Deleted Value =", self.tail.data)
        self.tail = temp
        self.tail.next = self.head
    else:
        print("Deleted Value =", temp.next.data)
        temp.next = temp.next.next

def count(self):
    if self.head is None:
        print("No Linked List")
        return

    count = 0
    temp = self.head

    while True:
        count += 1
        temp = temp.next
```

```

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return

        temp = self.head

        print("Circular Linked List:")

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("(Head)")

    def display_head_tail(self):
        if self.head is None:
            print("Circular Linked List is empty.")
        else:
            print("Head =", self.head.data)
            print("Tail =", self.tail.data)

c11 = CircularLinkedList()
while True:
    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create a Circular LL")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific position")
    print("5. Delete at beginning")
    print("6. Delete at end")
    print("7. Delete at specific position")
    print("8. Count no. of nodes")
    print("9. Traverse / Display the elements")

```

```

print("10. Display Head and Tail")
print("11. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
    cll.create()
elif choice == 2:
    cll.insert_beginning()
elif choice == 3:
    cll.insert_end()
elif choice == 4:
    cll.insert_position()
elif choice == 5:
    cll.delete_beginning()
elif choice == 6:
    cll.delete_end()
elif choice == 7:
    cll.delete_position()
elif choice == 8:
    cll.count()
elif choice == 9:
    cll.display()
elif choice == 10:
    cll.display_head_tail()
elif choice == 11:
    print("Program exited.")
    break

else:
    print("Invalid choice. Please try again.")

```

Output:

```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/lab 4/cll.py =====

----- CIRCULAR LINKED LIST -----
1. Create a Circular LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Delete at beginning
6. Delete at end
7. Delete at specific position
8. Count no. of nodes
9. Traverse / Display the elements
10. Display Head and Tail
11. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 27
Enter data: 27
Enter data: 18
Circular Linked List created successfully.

----- CIRCULAR LINKED LIST -----
1. Create a Circular LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Delete at beginning
6. Delete at end
7. Delete at specific position
8. Count no. of nodes
9. Traverse / Display the elements
10. Display Head and Tail
11. Exit
Enter your choice: 5
Deleted Value = 27

```

```
----- CIRCULAR LINKED LIST -----
1. Create a Circular LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Delete at beginning
6. Delete at end
7. Delete at specific position
8. Count no. of nodes
9. Traverse / Display the elements
10. Display Head and Tail
11. Exit
Enter your choice: 10
Head = 27
Tail = 18

----- CIRCULAR LINKED LIST -----
1. Create a Circular LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Delete at beginning
6. Delete at end
7. Delete at specific position
8. Count no. of nodes
9. Traverse / Display the elements
10. Display Head and Tail
11. Exit
Enter your choice: 11
Program exited.
```

>>>

EXPLANATION:

- The program defines a Node class with data and next to store the value and link to the next node.
- The Circular LinkedList class maintains two references: head for the first node and tail for the last node.
- In create(), the user enters the number of nodes and their data. Each node is added to the list, and tail.next is always connected back to head.

- `insert_beginning()` creates a node before the current head and then updates `tail.next` so the circular connection is maintained.
- `insert_end()` adds a node after the current tail, makes it the new tail, and connects it back to head.
- `insert_position()` traverses the list up to the required position and inserts the new node between existing nodes. If inserted at the end, tail is updated.
- `delete_beginning()` removes the first node by moving head to the next node and reconnecting `tail.next` to the new head.
- `delete_end()` traverses to the node before tail, removes the last node, and makes that node the new tail.
- `delete_position()` finds the required node and bypasses it by changing the previous node's next. If the deleted node is the tail, tail is updated.
- `count()` starts from head and keeps traversing until it reaches head again,
- `display()` prints every node starting from head and stops when the traversal reaches head again, avoiding an infinite loop.
- `display_head_tail()` directly prints the data stored in the head and tail nodes.
- The while True loop creates a menu-driven program where the user selects an operation using choice.
- The corresponding if-elif statements call the selected function.
- Selecting 11 executes break and terminates the program.
- The key feature of the code is that `tail.next = head`, which keeps the linked list circular.

ERROR TABLE:

S.no	Error Obtained	Rectification
1	Infinite loop during traversal/display because a circular linked list does not have a None at the end.	Use if temp == self.head: break to stop traversal when the pointer reaches the head again.
2	Incorrect circular connection after insertion or deletion if tail.next is not updated.	Maintain self.tail.next = self.head after every insertion or deletion so that the last node always points back to the first node.

Lab-5

1.Aim: Write a python program to implement a stack using an Array. Implement the stack operations such as push, pop, peak and also display the stack elements.

PROGRAM: Stack using an Array

stack using arrays.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 5/stack using arrays.py (3.14.2)

File Edit Format Run Options Window Help

```
class StackArray:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            for i in range(self.top, -1, -1):
                print(self.stack[i])

size = int(input("Enter size of stack: "))
s = StackArray(size)

while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
```

```

print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    item = int(input("Enter element to push: "))
    s.push(item)
elif choice == 2:
    s.pop()
elif choice == 3:
    s.peak()
elif choice == 4:
    s.display()
elif choice == 5:
    print("Exiting program...")
    break
else:
    print("Invalid choice!. Try again.")

```

Output:

```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)]
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/lab 5/stack using arrays.py ===
Enter size of stack: 5

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 15
15 pushed into the stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
15 popped from the stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:

```

EXPLANATION:

- class StackArray: : Defines a class to implement the stack.
- __init__(size) : Creates the stack and sets its initial values.
- self.stack = [None] * size : Creates an array with empty positions.
- self.top = -1 : Sets top to -1 because the stack is initially empty.
- push(item) : Adds an element to the stack.
 - o Checks if the stack is full.
 - o Increases top by 1.
 - o Stores the item at stack[top].
- pop() : Removes the top element.
 - o Checks if the stack is empty.
 - o Gets the element at stack[top].
 - o Removes it by setting it to None.
 - o Decreases top by 1.
- peek() : Shows the top element without removing it.
- display() : Uses a for loop to print elements from top to 0.
- size = int(input(...)) : Gets the stack size from the user.
- s = StackArray(size) : Creates a stack object.
- while True : Continuously displays the menu.
- choice : Takes the user's operation choice.
- LIFO principle : The last element pushed is the first element popped.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1	IndexError may occur if an element is accessed outside the array limit in the Array Stack.	Check whether the stack is full before push() and empty before pop() using top.

2.Aim: Write a python program to implement a stack using an Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

Program: Stack using an Linked List.

stack using linked list.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 5/stack using linked list.py (3.14.2)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class StackLinkedList:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            popped = self.top.data
            self.top = self.top.next
            print(popped, "popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next

sll = StackLinkedList()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
```



```
--- STACK USING LINKED LIST ---
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice: 3
```

```
Top element: 40
```

```
--- STACK USING LINKED LIST ---
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice: 5
```

```
Exiting program...
```

EXPLANATION:

- Node(data) : Creates a node and stores the element in data.
- next = None : Initially, the node has no link.
- top = None : Initializes an empty stack.

Push:

- new_node = Node(data) : Creates a new node.
- new_node.next = self.top : Links it to the current top.
- self.top = new_node : Makes it the new top.
- Process: Create : Link : Update top.

Pop:

- Checks self.top is None for underflow.
- popped = self.top.data : Gets the top value.
- self.top = self.top.next : Moves top to the next node.
- Process: Get value : Move top : Remove old top.

Peek:

- self.top.data : Displays the top value without removing it.

Display:

- temp = self.top : Starts from top.
- while temp : Traverses the nodes.
- temp = temp.next : Moves to the next node.

Main Program:

- SLL = Stack LinkedList() : Creates the stack.
- choice : Selects Push, Pop, Peek, Display, or Exit.
- while True : Repeats the menu.
- break : Exits the program.
- LIFO: Last inserted : First removed.

ERROR TABLE:

s.no	Error Obtained	Rectification
1.	Attribute Error may occur in Linked List Stack if self.top.data is accessed when top is None.	Check if self.top is None before performing pop() or peek().

